Why Data Cleaning Matters: Methods and Tools in R

Data Analysis for Decision-Making (SS 2026)

Cagan Goksay (Matriculation Nr.: 401004542)

May 16, 2026

Data cleaning is essential in data analysis as it ensures accuracy, consistency, and reliability of results, and R provides powerful methods and tools to efficiently detect, correct, and preprocess imperfect datasets.

1. Introduction: Bundesliga 23/24 Team & Player Stats

In modern data analysis, the quality of insights heavily depends on the quality of the underlying data. Raw datasets are often incomplete, inconsistent, or contain errors that can significantly affect analytical results. Therefore, data cleaning is a crucial step before performing any meaningful analysis.

This project focuses on a football dataset that includes performance-related variables such as expected assists, actual assists, minutes played, and match participation. The objective is to demonstrate how data cleaning and preprocessing techniques in R can improve data reliability and enable more accurate interpretations.

2. Dataset Overview

The dataset used in this project contains multiple variables related to player performance. These include:

- Rank
- Player
- Team
- Expected Assists

- Actual Assists
- Minutes Played
- Matches Played
- Country

The dataset provides a foundation for analyzing player creativity and performance efficiency by comparing expected assists (a predictive metric) with actual assists (real outcomes).

3. Data Import and Initial Exploration

The first step in the analysis is importing the dataset into R:

```
my_data <- read.csv("ASSIST_DATA.csv")
md <- my_data
```

Basic functions such as `head()`, `tail()`, and `View()` are used to inspect the dataset:

- `head(md)` displays the first rows
- `tail(md)` displays the last rows
- `View(md)` opens the dataset in a spreadsheet format

These functions help to quickly understand the structure and detect obvious issues.

4. Data Manipulation with tidyverse

The project uses the **tidyverse** package, which provides powerful tools for data manipulation.

```
#install.packages("tidyverse")
library("tidyverse")
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.0     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.1
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

A filtering operation is applied:

```
md |>
  select(Player, Team, Country) |>
  filter(Country == "GER" & Team == "Borussia Dortmund") |>
  arrange(Player)
```

```
                 Player              Team Country
1        Alexander Meyer Borussia Dortmund     GER
2  Antonios Papadopoulos Borussia Dortmund     GER
3               Emre Can Borussia Dortmund     GER
4           Felix Nmecha Borussia Dortmund     GER
5           Hendry Blank Borussia Dortmund     GER
6          Julian Brandt Borussia Dortmund     GER
7          Karim Adeyemi Borussia Dortmund     GER
8           Kjell Wätjen Borussia Dortmund     GER
9             Marco Reus Borussia Dortmund     GER
10           Marius Wolf Borussia Dortmund     GER
11          Mats Hummels Borussia Dortmund     GER
12       Niclas Füllkrug Borussia Dortmund     GER
13    Nico Schlotterbeck Borussia Dortmund     GER
14            Niklas Süle Borussia Dortmund     GER
15          Ole Pohlmann Borussia Dortmund     GER
16          Samuel Bamba Borussia Dortmund     GER
17     Youssoufa Moukoko Borussia Dortmund     GER
```

This code:

- Selects relevant columns
- Filters players based on country and team
- Sorts the results alphabetically

This demonstrates how to extract meaningful subsets from a larger dataset.

5. Understanding Data Structure

```
glimpse(md)
```

```
Rows: 494
Columns: 8
$ Rank             <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16~
$ Player           <chr> "Alejandro Grimaldo", "David Raum", "Kevin Stöger", "~
$ Team             <chr> "Bayer 04 Leverkusen", "RB Leipzig", "VfL Bochum", "B~
$ Expected.Assists <chr> "11.1", "10.7", "9.8", "9.7", "9.5", "9.4", "9.3", "9~
$ Actual.Assists   <int> 13, 8, 9, 11, 11, 9, 6, 7, 11, 7, 9, 11, 8, 6, 6, 4, ~
$ Minutes          <int> 2785, 2743, 2678, 2377, 2142, 2358, 2185, 2594, 2673,~
$ Matches          <int> 33, 31, 32, 32, 27, 32, 28, 34, 32, 32, 31, 32, 32, 3~
$ Country          <chr> "ESP", "GER", "AUT", "GER", "GER", "FRA", "GER", "GER~
```

```
class(md$Country)
```

```
[1] "character"
```

```
unique(md$Country)
```

```
 [1] "ESP" "GER" "AUT" "FRA" "NED" "ITA" "CRO" "ALG" "ENG" "SUI" "CZE" "GUI"
[13] "DEN" "BEL" "EGY" "FIN" "JPN" "ARG" "BIH" "TUR" "HUN" "POR" "NGA" "CAN"
[25] "TOG" "GHA" "KVX" "MAR" "BRA" "NOR" "SWE" "KOR" "USA" "TUN" "SVK" "COL"
[37] "POL" "COD" "UKR" "ECU" "MLI" "SUR" "SVN" "CIV" "CMR" "BFA" "ARM" "LUX"
[49] "CRC" "SRB" "SYR" "ALB" "SCO" "MKD" "PHI" "SEN" "ISR" "BUL"
```

- `glimpse()` provides a compact overview of all variables
- `class()` identifies the data type (e.g., character, numeric)
- `unique()` shows all distinct values in a column

These steps are essential for identifying inconsistencies or incorrect data types.

6. Descriptive Statistics

Basic statistical analysis is performed:

```r
mean(md$Actual.Assists, na.rm = TRUE)
```

```
[1] 1.415822
```

This calculates the average number of actual assists while ignoring missing values. It gives a quick understanding of overall player performance.

7. Handling Missing Values

Missing values (NA) can distort analysis results. Therefore, they must be handled properly.

Removing Missing Values

This removes rows with missing values.

Identifying Missing Data

```r
md |>
  select(Rank, Player, Expected.Assists, Actual.Assists) %>%
  filter(!complete.cases(.))
```

```
  Rank       Player Expected.Assists Actual.Assists
1  417 Cagan Goksay                              NA
```

This identifies rows that contain missing values.

8. Data Cleaning and Imputation

To handle missing values:

```r
md |>
  select(Rank, Player, Team, Expected.Assists, Actual.Assists) %>%
  filter(!complete.cases(.)) %>%
  mutate(Team = ifelse(is.na(Team) | Team == "" | Team == " ", "none", Team)) %>%
  mutate(Expected.Assists = ifelse(is.na(Expected.Assists) | Expected.Assists == "" | Expecte
  mutate(Actual.Assists = ifelse(is.na(Actual.Assists) | Actual.Assists == "" | Actual.Assist
```

```
  Rank       Player Team Expected.Assists Actual.Assists
1  417 Cagan Goksay none                0              0
```

- Missing character values → replaced with "none"
- Missing numeric values → replaced with 0

Note: For demonstration purposes, missing numaric values were temporarily replaced with 0.

This ensures consistency and prevents errors in further analysis.

9. Detecting Duplicate Data

Duplicate records can bias results. The following code identifies duplicates:

```r
md_duplicates <- md %>%
  filter(duplicated(Player) | duplicated(Player, fromLast = TRUE)) %>%
  arrange(Player)


md_duplicates |>
    select(Rank, Player, Team, Expected.Assists, Actual.Assists)
```

```
  Rank         Player              Team Expected.Assists Actual.Assists
1  416 Thorgan Hazard Borussia Dortmund                0              0
2  416 Thorgan Hazard Borussia Dortmund                0              0
3  416 Thorgan Hazard Borussia Dortmund                0              0
```

This helps ensure that each player is represented correctly in the dataset.

10. Conclusion

This project highlights the importance of data cleaning in the data analysis process. Using R and tidyverse tools, the dataset was explored, cleaned, and prepared for analysis. The cleaning process revealed that football performance datasets may contain incomplete player statistics and duplicated entries, which can directly affect comparative performance analysis.

Key takeaways include:

- Data cleaning improves accuracy and reliability
- Missing values must be handled carefully

- Duplicate data should be identified and removed
- Proper preprocessing enables meaningful insights

Overall, this project demonstrates how structured data preparation leads to more trustworthy and interpretable analytical results.

:::